Archive Digitization Tool

Software Project Lab 3 Technical Report

Submitted by
Md Yasir Arafat Araf
BSSE-1220
Institute of Information Technology
University of Dhaka

Supervised by
Dr. Sumon Ahmed
Associate Professor
Institute of Information Technology
University of Dhaka

Submission Date: 01 December, 2024



Letter of Transmittal

01 December, 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - **Archive Digitization Tool**

Sir,
I am submitting the technical report for our Software Project Lab 3, **Archive Digitization Tool.**
This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,

Md Yasir Arafat Araf
BSSE-1220
Institute of Information Technology
University of Dhaka

Supervisor's Signature

Acknowledgment

I am grateful to the Institute of Information Technology for giving me such an opportunity to work on “Archive Digitization Tool”. I would like to convey my gratitude to my supervisor, Dr. Sumon Ahmed, Associate Professor, Institute of Information Technology, University of Dhaka, for providing me with guidelines about how I can prepare this analysis document. Lastly, I would like to thank my classmates. They have always been helpful and provided valuable insights from time to time.

Abstract

This document provides a detailed overview of the software requirements, specifications, architectural design, and testing processes for the "Archive Digitization Tool." This tool is a comprehensive web application designed to extract Bangla text from images using advanced machine learning techniques. The application leverages the YOLO model for precise text detection and integrates Optical Character Recognition to accurately extract textual content from complex layouts.

One of the key strengths of this tool is its ability to preserve the original layout structure, including spacing, alignment, and font styles, in the digitized format. This project is motivated by the lack of robust tools for Bangla text processing, particularly those that address the complexities of digitizing Bangla documents with non-standard layouts. By providing an efficient and scalable solution, this tool not only simplifies the process of digitization but also contributes to the creation of a vast digital resource pool. Such resources can be invaluable for furthering research and development in the field of Bangla language processing and machine learning.

Letter of Transmittal **1**
Acknowledgment **2**
Abstract **3**
1. Introduction **7**
1.1 Motivation 7
1.2 Scope 7
1.3 Project Timeline 8
Figure -1 : Timeline for Archive Digitization Tool 8
1.4 Purpose of Document 8
2. Project Description 8
2.1 Quality Function Deployment 9
2.1.1 Normal Requirements 9
2.1.2 Expected Requirements 9
2.1.3 Exciting Requirements 9
2.2. Usage Scenario 10
Layout Detection 10
Text Recognition with OCR 10
Layout Preservation 10
Export Options 10
3. Scenario Based Modelling **11**
3.1 Use Case Diagram 11
3.1.1 Archive Digitization Tool 11
Figure -2 : Use Case Level - 0 11
3.1.2 Modules of Archive Digitization Tool 12
Figure -2 : Use Case Level - 1(Modules) 12
3.1.2.1 Image Upload 12
3.1.2.2 Layout Detection 13
3.1.2.3 Text Extraction 13
3.1.2.4 Layout Preservation 13
3.1.2.5 Result display 13
4. Class-Based Modeling **13**
4.1 Analysis Classes 13
4.2 Class cards 14
4.3 Class Responsibility and Collaboration Diagram 17
5. Architectural Design **18**
5.1 Architectural Context Diagram 18
5.2 Defining Archetypes 18

Figure -6 : Pipe and Filter Architecture 19
5.3 Top Level Components 19
Figure - 7 : Top Level Components 19
5.4 Instantiation of the Components: 20
Figure - 8 : Instantiation of the Components 20
5.4.1 Layout Detector 20
5.4.2 OCR Operations 21
6. Preliminary Test Plan 21
6.1 High Level Description of Testing Goals 21
6.2 Test Cases 21
7. Conclusion 25
References: 26

List Of Figures

Figure-01: Timeline for Archive Digitization Tool 08
Figure-02: Use Case Level 0 11
Figure-03: Use Case Level 1 12
Figure-04: CRC Diagram 17
Figure-05: Architectural Context Diagram 18
Figure 06: Pipe and Filter Architecture 19
Figure-07: Top Level Components 19
Figure-08: Instantiation of the Components 20

List of Tables

Table-1: CRC Card (ImageUpload) 13
Table-2: CRC Card (LayoutDetector) 14
Table-3: CRC Card (Model) 14
Table-4: CRC Card (OCRProcessor) 15
Table-5: CRC Card (LayoutPreserver) 15
Table-6: CRC Card (Exporter) 16

1. Introduction

Archive Digitization Tool is an web application that provides a system for extracting Bangla text from images, utilizing cutting-edge machine learning techniques. The system will first analyze the structure of the image to detect and locate areas containing text using a YOLO (You Only Look Once) model and apply OCR on it. The system is designed to handle various layouts, fonts, and formats, ensuring accurate text extraction. Once processed, users will be able to preview and export the extracted text in various formats, including plain text, PDFs, or formatted documents like Word and Excel. This solution addresses the gap in tools for processing Bangla-language documents and provides a robust and efficient method for digitizing and extracting Bangla text.

1.1 Motivation

The motivation behind working with this project is the lack of an efficient tool to digitize Bangla documents from images, especially when dealing with complex layouts. While current technologies offer solutions for text extraction, specialized tools for Bangla are still limited. This project aims to enhance text extraction capabilities for accurate Bangla text recognition, supporting the digitization of Bangla documents across various fields. Besides it is not feasible to preserve or manage the myriad of data physically, hence digitalizing them provides a means of storing them.

1.2 Scope

The scope of **Archive Digitization Tool** is given below:

- It will be a website
- It will detect layouts from images
- Implement OCR for Bangla text recognition from localized areas
- Recreate the exact layout of the input image in the digitized output.

1.3 Project Timeline

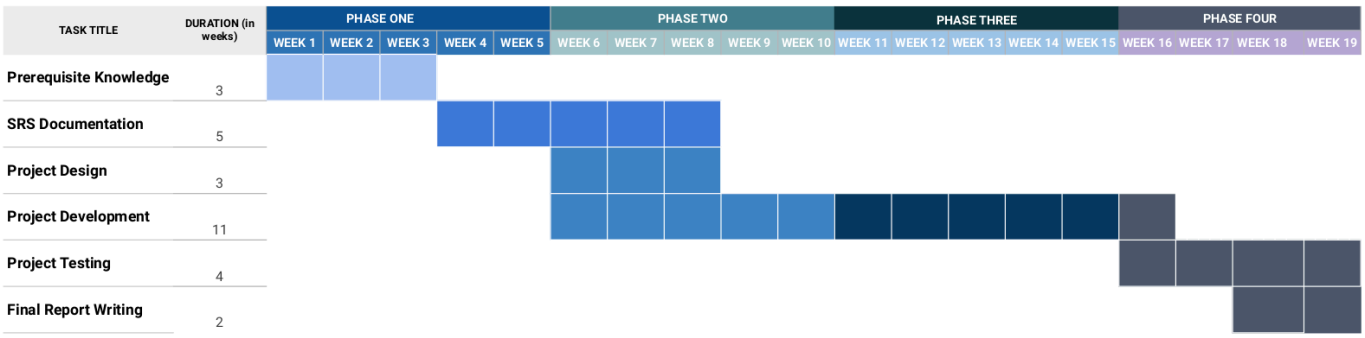


Figure -1 : Timeline for Archive Digitization Tool

1.4 Purpose of Document

The purposes of this document are-

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

2. Project Description

The following depicts the overview of the quality function deployment of Archive Digitization Tool. The journey towards defining the project's Quality Function Deployment (QFD) began with an in-depth review of relevant research literature, notably the paper authored by Mr. Asif Isthiaq and Ms. Najoa Asreen Saif, providing valuable insights into the domain. This project simplifies the process of digitizing Bangla content, enabling users to process handwritten, printed, or scanned documents efficiently. It helps organizations and individuals archive, search, and edit Bangla documents, addressing the lack of robust solutions for Bangla text processing and significantly improving workflow productivity.

2.1 Quality Function Deployment

Quality Function Deployment (QFD) is a technique that translates the needs of the customer into technical requirements for the software. With respect to this project, the following requirements are identified

2.1.1 Normal Requirements

These are the basic requirements that users expect as part of the application's functionality.

1. **Layout Detection** : The system will segment the areas containing text and images.
2. **Text Detection**: The system accurately detects Bangla text in uploaded images.
3. **OCR Processing**: Extract Bangla text from the detected regions with reasonable accuracy.
4. **Export Options**: Save extracted text as plain text, PDF or Word documents.

2.1.2 Expected Requirements

These requirements elevate the user experience and provide robust performance.

1. **Layout Preservation**: Recreate the exact layout of the input image in digitized outputs.
2. **Font Adaptability**: Support for various Bangla fonts.
3. **User Friendly Interface :** Easily understandable interface for uploading images , and displaying and exporting output.
4. **Classify Layouts** : Classify the layouts based on their label(image, paragraphs)

2.1.3 Exciting Requirements

These are features that delight users and go beyond expectations.

1. **Editable Outputs**: Allow users to edit extracted text within the application before exporting.
2. **Real-Time Preview**: Allow users to see the extracted text alongside the original image before export.

2.2. Usage Scenario

Image Upload

Initially, user will upload an image containing texts and images to digitize. This works as the initiating event for the tool.

Layout Detection

The system analyzes the uploaded image to identify and segment text-containing regions using a layout detection algorithm, such as YOLO. It generates bounding boxes or coordinates for these regions, enabling precise localization of text areas. This step ensures the accurate segmentation of the image's structure, even for complex layouts with mixed content.

Text Recognition with OCR

The segmented text areas are processed by the OCR module to extract Bangla text. The OCR is trained to handle various fonts, sizes, and handwritten scripts, ensuring high accuracy. Preprocessing techniques, such as noise reduction, may be applied to enhance text readability in challenging images.

Layout Preservation

Extracted text is placed in the output document according to the coordinates from the layout detection step. This process ensures the final output matches the original image’s structure, maintaining alignment, spacing, and positioning. The result is a faithful digital reproduction of the input image.

Export Options

The system provides multiple export options, including plain text, PDF and Word. Users can select their preferred format, and the system generates a downloadable file while preserving the layout. These options allow users to integrate the digitized text into various workflows seamlessly.

3. Scenario Based Modelling

Scenario-based modeling is a technique used in software development to create models that illustrate how a system operates or responds to different real-world situations or user interactions. These scenarios, often derived from events or conditions, provide a clear understanding of the system's behavior in various contexts. By simulating these scenarios, developers can identify potential challenges, make better design choices, and ensure the software aligns with user needs across diverse use cases.

3.1 Use Case Diagram

A use case diagram is a simple and intuitive way to visualize how a system interacts with its users or other systems. It shows the key actions (use cases) that the system performs and identifies the various users or external systems (actors) involved. These diagrams help clarify the scope of a system's functionality, making it easier to understand how users will interact with the system and what features it needs to support. Use case diagrams are often used to capture functional requirements and offer a high-level overview of system behavior.

3.1.1 Archive Digitization Tool

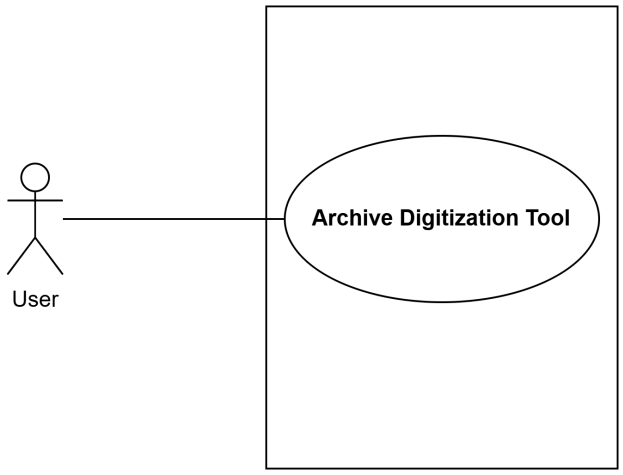


Figure -2 : Use Case Level - 0

Primary actors: User

Secondary actors: None

Goal in Context: The diagram shown above represents the whole Archive Digitization Tool

3.1.2 Modules of Archive Digitization Tool

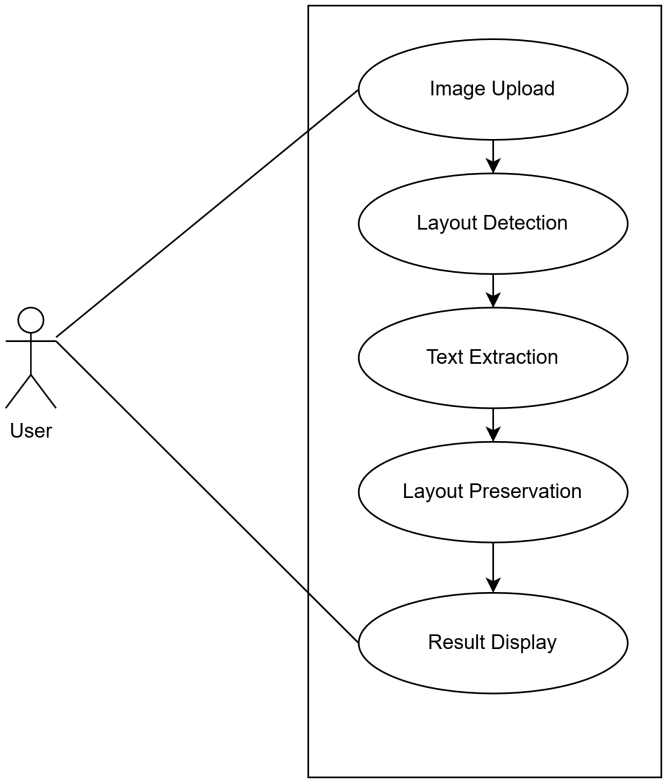


Figure -3 : Use Case Level - 1(Modules)

Primary actors: User

Secondary actors: None

Goal in context: The diagram shown above shows all the modules of the system. The system consists of 5 modules

3.1.2.1 Image Upload

This module serves as the entry point for this tool's functionality. It validates the file

format and prepares the image for subsequent operations, ensuring compatibility with the system's layout detection and text extraction processes.

3.1.2.2 Layout Detection

This module identifies and segments text-containing regions within the uploaded image. Using advanced algorithms like YOLO, it generates bounding boxes or coordinates for these areas, enabling precise localization of text blocks for further processing.

3.1.2.3 Text Extraction

The text extraction module processes the segmented regions and applies OCR to convert the Bangla text within each area into editable digital text. It ensures high accuracy in recognizing various fonts, sizes, and handwritten scripts.

3.1.2.4 Layout Preservation

This module reconstructs the extracted text by placing it in the same spatial arrangement as in the original image. By using the coordinates generated during layout detection, it ensures that the output document matches the layout of the input image.

3.1.2.5 Result display

This module displays the extracted text and layout in a preview interface for user review. It provides options to export the final output in multiple formats, such as plain text, PDF, Word, or Excel, allowing users to choose the format that best suits their needs.

4. Class-Based Modeling

4.1 Analysis Classes

After identifying nouns from the scenario, I have filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes:

1. ImageIO
2. Model
3. LayoutDetector
4. OCRProcessor
5. LayoutPreserver
6. Exporter

4.2 Class cards

The class cards of this project is given below:

ImageUpload	
Attributes	**Methods**
-inputImage	-uploadImage() -validateImage()
Responsibilities	**Collaborators**
• input image from user • Entry point for the application	LayoutDetector

Table-1 : Image Upload

LayoutDetector	
Attributes	**Methods**
-layoutRegions -boundingBoxes	-detectLayout() -getLayoutRegions() -identifyRegions() -generateBoundingBoxes() -classifyRegions() -getClassifiedRegions() -classifyBoundingBoxes() -mergeOverlappingBoxes() -getLargestBoundingBox() -getBoundingBoxArea()
Responsibilities	**Collaborators**
● Identify text-containing regions ● Generate bounding boxes ● Provide structured output (e.g., bounding boxes or coordinates) for the detected layout.	ImageUpload, OCRProcessor

Table-2 : LayoutDetector

Model	
Attributes	**Methods**
-model	+loadModel() -train() +predict()

Responsibilities	**Collaborators**
• train the model • make predictions	OCRProcessor

Table-3 : Model

OCRProcessor	
Attributes	**Methods**
-processed_images -extracted_text	-perform_ocr() -extract_text() -extract_paragraphs() -extract_images()
Responsibilities	**Collaborators**
• Perform OCR on segmented regions to extract text. • Extract textual content, tables and images from classified bounding boxes.	Model, LayoutDetector

Table-4 : OCRProcessor

LayoutPreserver	
Attributes	**Methods**
-layout_coordinates	-map_content_to_coordinates() -apply_alignment() -apply_spacing()

Responsibilities	**Collaborators**
● Map the extracted texts to the position of original image ● Formatting aligning and spacing	OCRProcessor, LayoutDetector, Exporter

Table-5 : LayoutPreserver

Exporter	
Attributes	**Methods**
-output_format -file_path	-set_output_format() -export() -edit()
Responsibilities	**Collaborators**
● Select the output format ● Export in the desired format ● Edit before exporting	LayoutPreserver

Table- 6: Exporter

4.3 Class Responsibility and Collaboration Diagram

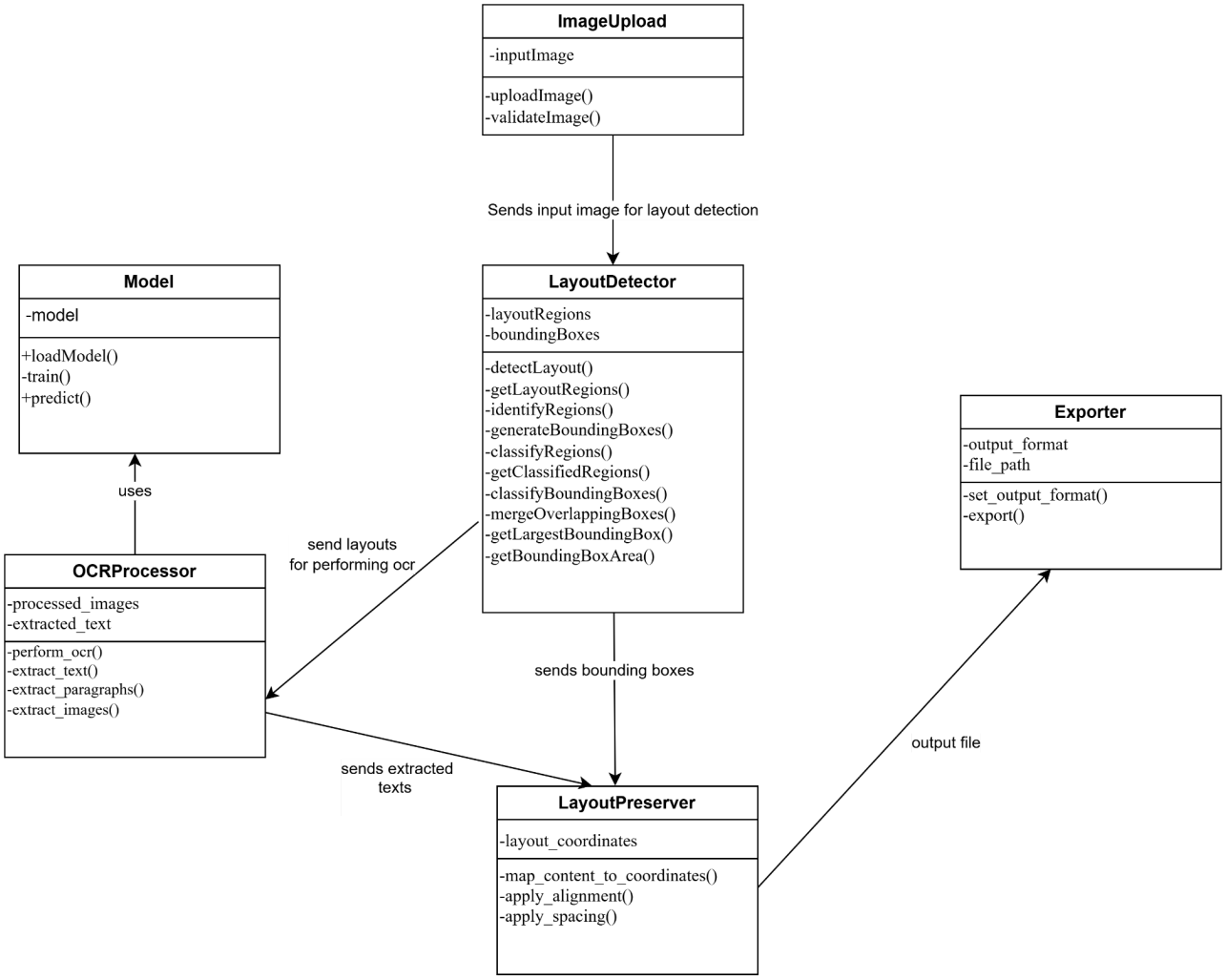


Figure -4 : CRC Diagram

5. Architectural Design

Architectural design is the high-level structure of a software system, defining how its components are organized, interact, and function together to achieve the system's goals. It serves as a blueprint for both the system and the project, offering a shared understanding for stakeholders, developers, and other contributors.

5.1 Architectural Context Diagram

Any general user can use the tool. There are no subordinate, superordinate or

peer-level systems. The architectural context diagram for the tool is given below:

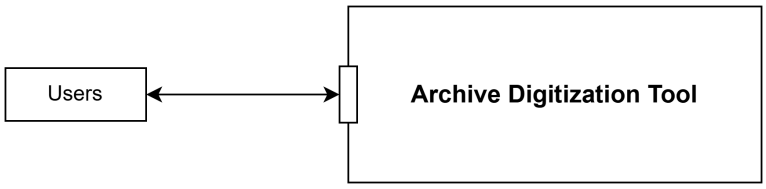


Figure - 5 : Architectural Context Diagram

5.2 Defining Archetypes

An archetype is a class or pattern that represents a core abstraction that is critical to the design of an architecture for the target system. In general, relatively small set archetypes are required to design even relatively complex systems.In many cases, archetypes can be derived by examining the analysis classes defined as part of the requirements model. In our cases, the archetypes are:

1. Pipe and Filter Architecture

Pipe and Filter Architecture

The Pipe and Filter architecture is a design pattern that divides a process into a series of distinct steps, called filters, linked by channels known as pipes. Each filter is dedicated to a particular processing function, whether it involves transforming, validating, or aggregating data. Data flows through these filters via pipes, which carry the output of one filter to the input of another.

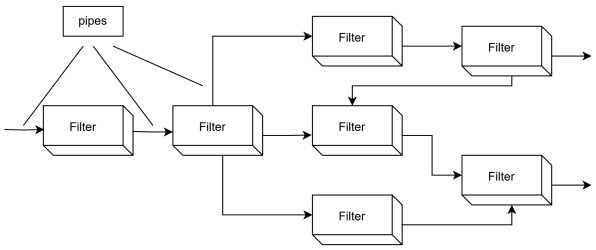


Figure -6 : Pipe and Filter Architecture

- **Filters**: Each filter carries out a specific, standalone task, whether it be transforming, validating, or processing data before forwarding it. In the setup, there are two levels

of filters; the first one processes data from the pump and hands it to the second filter, which continues the processing before passing it on.

- **Pipes**: Pipes serve as the channels for data movement between filters, linking each component in sequence and ensuring a smooth transfer of data. In diagrams, pipes are depicted as arrows connecting the components.

5.3 Top Level Components

Overall architectural structure with top level components is illustrated below:

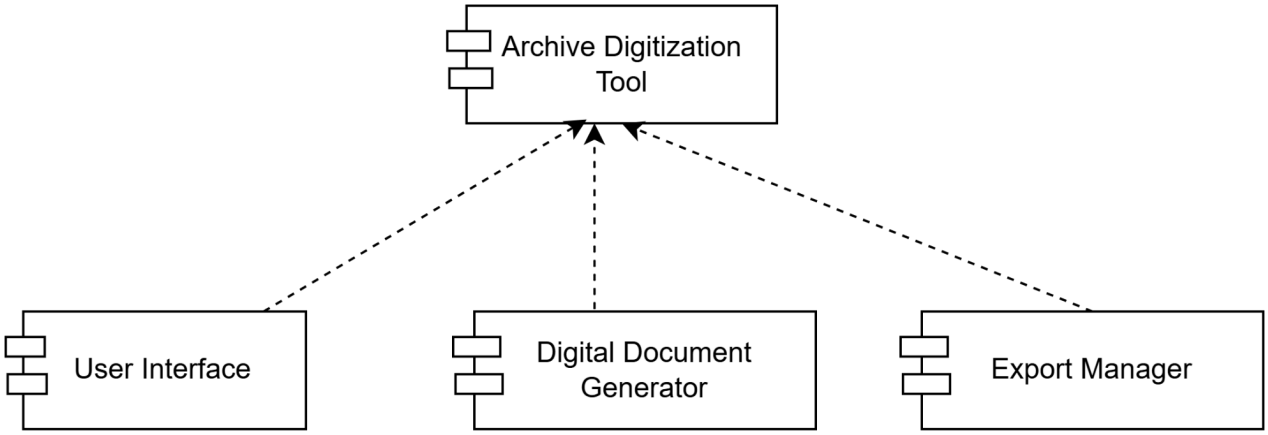


Figure - 7 : Top Level Components

5.4 Instantiation of the Components:

The components identified in Top Level Components are broad. They need to be refined into smaller components. So we apply architecture on the components to make a maintainable collection of components.

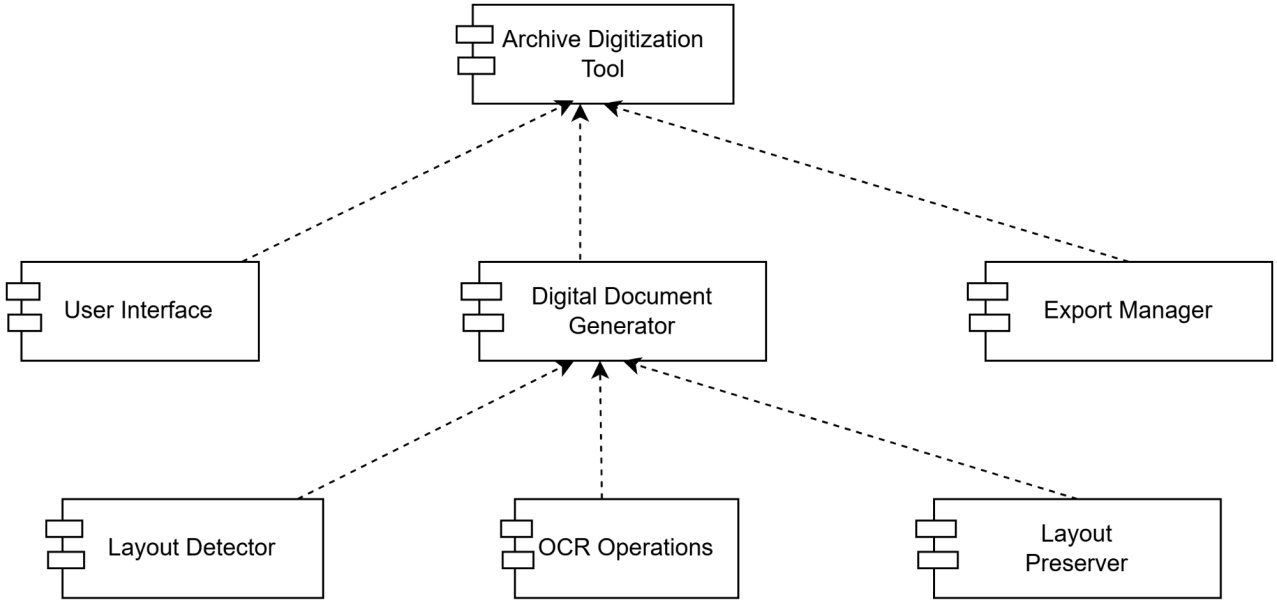
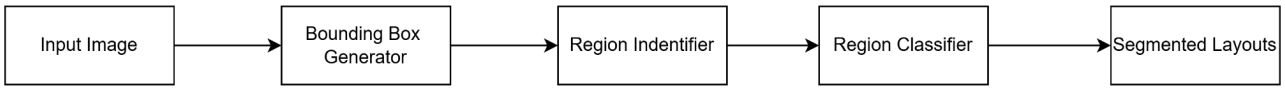


Figure - 8 : Instantiation of the Components

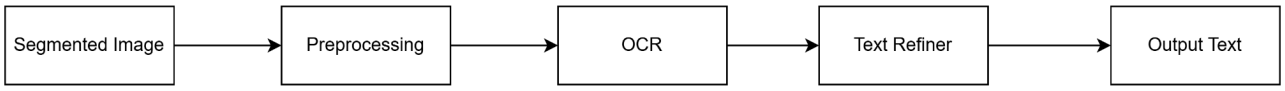
5.4.1 Layout Detector

Pattern: Pipe & Filter Architecture



5.4.2 OCR Operations

Pattern: Pipe & Filter Architecture



6. Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

6.1 High Level Description of Testing Goals

Testing goals for this tool is represented below from a high level:

- To demonstrate that the system meets its requirements.
- To ensure that the uploaded documents are processed accurately, with correct layout detection and segmentation.
- To ensure that OCR extracts text with a high degree of accuracy
- To ensure that the system preserves the original layout structure.
- To ensure that the exported files (PDF, Word, Excel) match the processed content and layout accurately.
- To ensure that the system runs smoothly and performs efficiently, even with large or complex documents.

6.2 Test Cases

Test cases for the testing of this tool are documented below:

Test ID: T1
Test Case: Image Upload Test
Input Test Data: Images of varying formats and sizes (JPEG, PNG, TIFF, >10MB files).
Steps to Be Executed:

1. Upload images via the interface.
2. Verify if valid images are accepted.
3. Observe the system's response to unsupported or corrupted files.

Expected Result:

- Valid images are successfully uploaded.
- Unsupported or corrupted files display an appropriate error message.

Target Item: File upload module.

Pass/Fail: Pending

Test ID: T2
Test Case: Layout Detection Test
Input Test Data: Documents with single-column, multi-column, and mixed layouts.
Steps to Be Executed:
Upload a document with a known layout.
Perform layout detection.
Compare detected regions with expected regions.
Expected Result:
Bounding boxes align with content.
Regions are accurately classified (e.g., text, image, table).
Target Item: Layout detection process.
Pass/Fail: Pending

Test ID: T3

Test Case: OCR Accuracy Test
Input Test Data: Text-heavy documents, noisy images, and rotated text samples.
Steps to Be Executed:
Upload the document.
Perform OCR on detected text regions.
Compare extracted text with the original document.
Expected Result:
Extracted text matches original content with minimal errors.
Rotated and noisy text are handled correctly.
Target Item: OCR processing module.
Pass/Fail: Pending

Test ID: T4
Test Case: Layout Preservation Test
Input Test Data: Documents with specific alignments, spacing, and font styles.
Steps to Be Executed:

1. Extract and process content from the document.
2. Compare the recreated layout with the original.

Expected Result:

- Alignment, spacing, and fonts are preserved in the processed layout.

Target Item: Layout preservation process.
Pass/Fail: Pending

Test ID: T5
Test Case: Export Functionality Test
Input Test Data: Processed text and layout from various documents.
Steps to Be Executed:

1. Choose an export format (PDF, Word, Excel).
2. Verify the generated file's content and layout fidelity.

Expected Result:

- Exported files retain the processed content and layout accurately.
- All export formats function as intended.

Target Item: Export functionality.

Test ID: T6
Test Case: Preserve Tables and Grid Layouts
Input Test Data: Documents containing complex tables with merged cells and grid layouts.
Steps to Be Executed:

Upload a document with complex table structures.

Process the document and attempt layout preservation.

Export the result and verify the table structure.

Expected Result:

Table structure, including merged cells and grids, is preserved in the output.

Exported files match the original layout accurately.

Target Item: Layout preservation and export modules.

Pass/Fail: Pending

Test ID: T7

Test Case: Detect Overlapping Regions

Input Test Data: Documents where text blocks or images overlap (layered text and images).

Steps to Be Executed:

1. Upload a document with overlapping text and image regions.
2. Perform layout detection.
3. Verify the detected bounding boxes for overlap handling.

Expected Result:

Overlapping regions are accurately segmented and classified.

Target Item: Region segmentation and bounding box generation.

Pass/Fail: Pending

Test ID: T15

Test Case: Detect Empty Regions

Input Test Data: Documents with large blank spaces or margins.

Steps to Be Executed:

- Upload a document with significant blank areas or non-content regions.
- Perform layout detection.
- Verify that blank regions are excluded or not misclassified.

Expected Result: Empty regions are ignored or excluded from detected layout regions.

Target Item: Region classification and filtering.

Pass/Fail: Pending

7. Conclusion

In this report, I have discussed all the necessary technical details to develop the “Archive Digitization Tool.” The scope, assumptions, Quality Function Deployment, and usage scenarios have been elaborated. Scenario-based modeling, data-based modeling, and class-based modeling have been performed to provide a comprehensive understanding of the system.

The architectural design and methodology have been discussed, outlining the high-level components and their interactions. Additionally, I have presented the testing goals and a detailed summary of the items and features to be tested. Figures and tables have been included to enhance clarity and provide a better understanding of the technical details.

With this document, I have aimed to clearly state the workflow and functionality of the entire system. I hope this report provides the reader with a thorough understanding of the design, implementation, and testing of this tool.

References:

[1] Diwan, Tausif, G. Anirudh, and Jitendra V. Tembhurne. "Object detection using YOLO: Challenges, architectural successors, datasets and applications." *multimedia Tools and Applications* 82.6 (2023): 9243-9275.

[2] Isthiaq, Asif, and Najoa Asreen Saif. "Ocr for printed bangla characters using neural network." *International Journal of Modern Education and Computer Science* 10.2 (2020): 19.